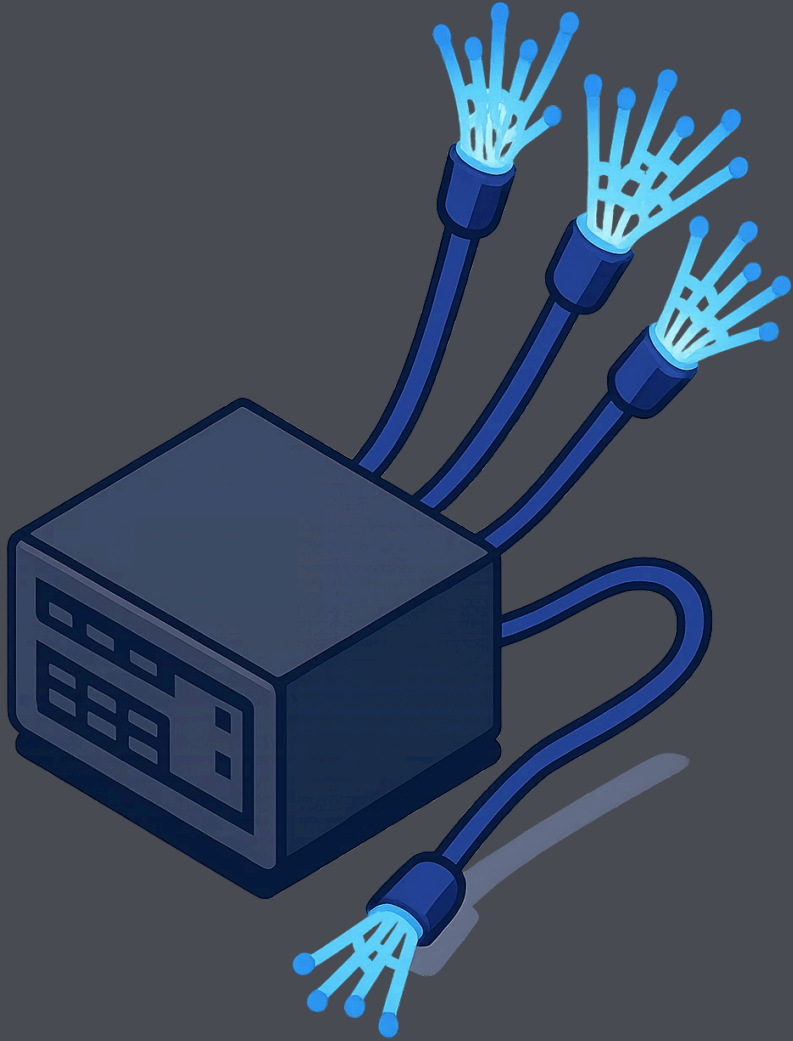




Day 2: Networking · Linux · Docker

From Zero Background to Running Containers in Production

Sami Selim

Instructor: DevOps Engineering

Agenda

01

Networking Fundamentals

How the internet actually works.

03

Linux Command Line Essentials

The 20 commands every DevOps engineer lives by.

05

What is Virtualization?

VMs explained.

07

Containers vs. VMs

The revolution.

09

Hands-On with Docker

Images, Containers, Dockerfiles.

02

IP Addresses, Ports & DNS

The language of the internet.

04

File Permissions & Process Management

06

Lab: Installing VMware & Ubuntu Desktop

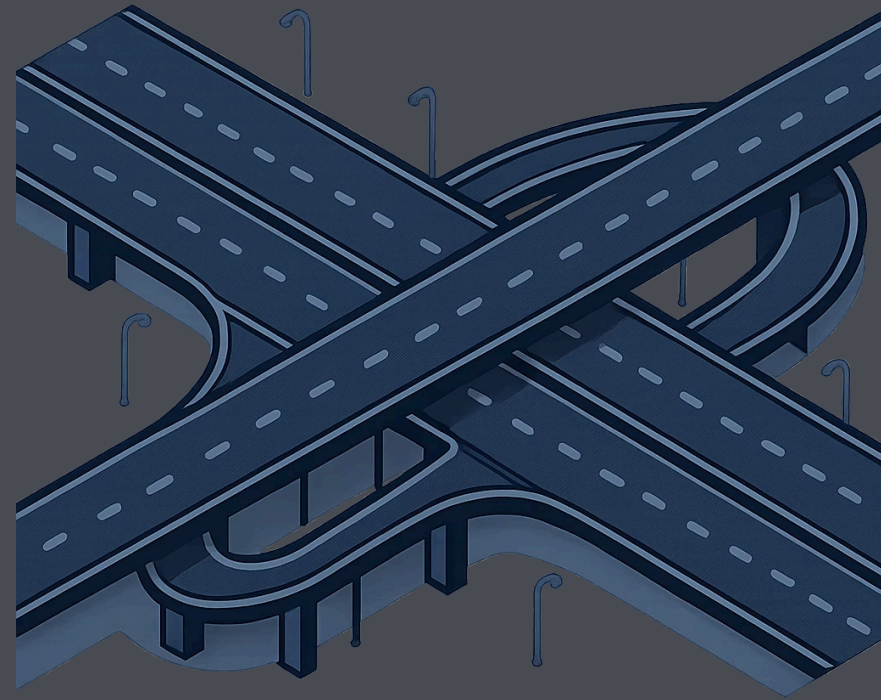
08

Docker Architecture Deep Dive

PART 1

Networking Fundamentals

How data travels from your screen to a server halfway around the world — and back — in milliseconds.



What is a Network?

A **network** is simply two or more devices connected together so they can exchange data.

Your home Wi-Fi is a small network. The Internet is the biggest network ever built.

Think of roads: a small town has a few streets connecting houses. The internet is a planet-wide highway system connecting billions of devices.

The Key Question

"How does a message sent from your laptop in Cairo reach a server located in a data center in Virginia, USA, in under 100 milliseconds?"

Network Devices: The Hardware Layer



NIC (Network Interface Card)

Every device has one. It is the physical piece that connects your machine to a network cable or Wi-Fi. It has a unique address burned in at the factory: the MAC Address.



Switch

A smart device inside your home or office that forwards packets to the correct device on your local network.



Router

The device connecting your local network to the outside world (internet). Your internet box at home is a Router.



Modem

Converts your ISP's signal (fiber, DSL) to digital signals your Router understands.

TCP vs. UDP

Two ways to send data

TCP — Transmission Control Protocol

Guarantees delivery. If a packet is lost, it is re-sent.

Like sending a Registered Mail letter — you get a receipt that it arrived.

Used for: Web (HTTP), Email, File Transfers.

UDP — User Datagram Protocol

Fire and forget. No guarantee of delivery.

Like shouting into a room — fast, but some words might be missed.

Used for: Live Video Streaming, Online Gaming, DNS.

IP Addresses

The Unique Home Address of Every Device

An **IP Address** is a logical label assigned to every networked device so data knows where to go.

IPv4 Format: 4 numbers separated by dots. Each number is 0–255.

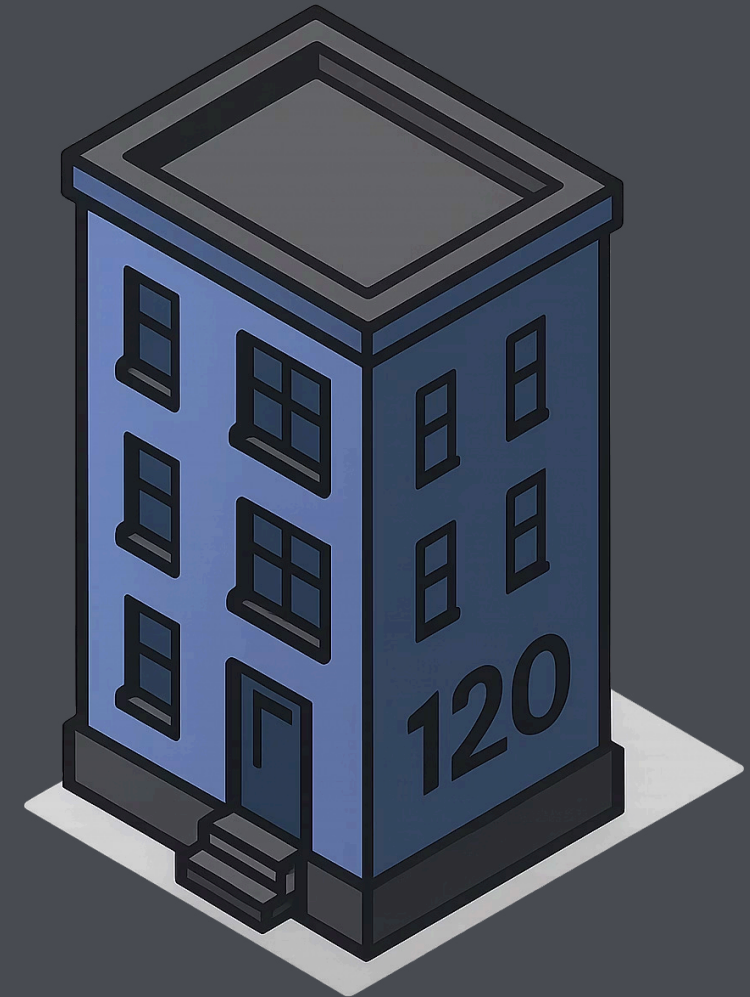
```
192.168.1.50
```

IPv6: The newer, much longer format.

```
2001:0db8:85a3::8a2e:0370:7334
```

Public vs Private IP

- **Private:** Used inside your local network (home, office). E.g., `192.168.x.x`
- **Public:** The unique address your Router presents to the whole internet.



Ports: The Apartment Number

An IP address is like the address of a **building**. A **Port** is like the apartment number inside that building.

One server can host 50 different services, served over 50 different ports.

The operating system uses the port number to forward incoming data to the correct running application.

Well-Known Ports

Port	Service
22	SSH (Remote Terminal)
80	HTTP (Web, unencrypted)
443	HTTPS (Web, encrypted)
5432	PostgreSQL Database
3306	MySQL Database
6379	Redis Cache

DNS: The Internet's Phone Book

The Problem: Humans can't remember `142.250.80.46`. We can remember `google.com`.

DNS (Domain Name System) acts as a translator. When you type `google.com`, your machine silently asks a DNS server: "What is the IP address for `google.com`?"

DNS replies: "It's `142.250.80.46`." Your browser then makes the actual connection to `142.250.80.46`.

Key Commands

```
nslookup google.com    # Lookup the IP of a domain
dig google.com A        # Advanced DNS query
```

Subnets and CIDR Notation

A **Subnet** divides a large network into smaller, isolated sections.

CIDR Notation expresses a subnet range compactly:

- `192.168.1.0/24` → All addresses from `192.168.1.0` to `192.168.1.255` (256 addresses).
- `/24` means the first 24 bits are the "network address", leaving 8 bits for host addresses.

This matters hugely in AWS, where VPCs and Subnets are critical infrastructure components.

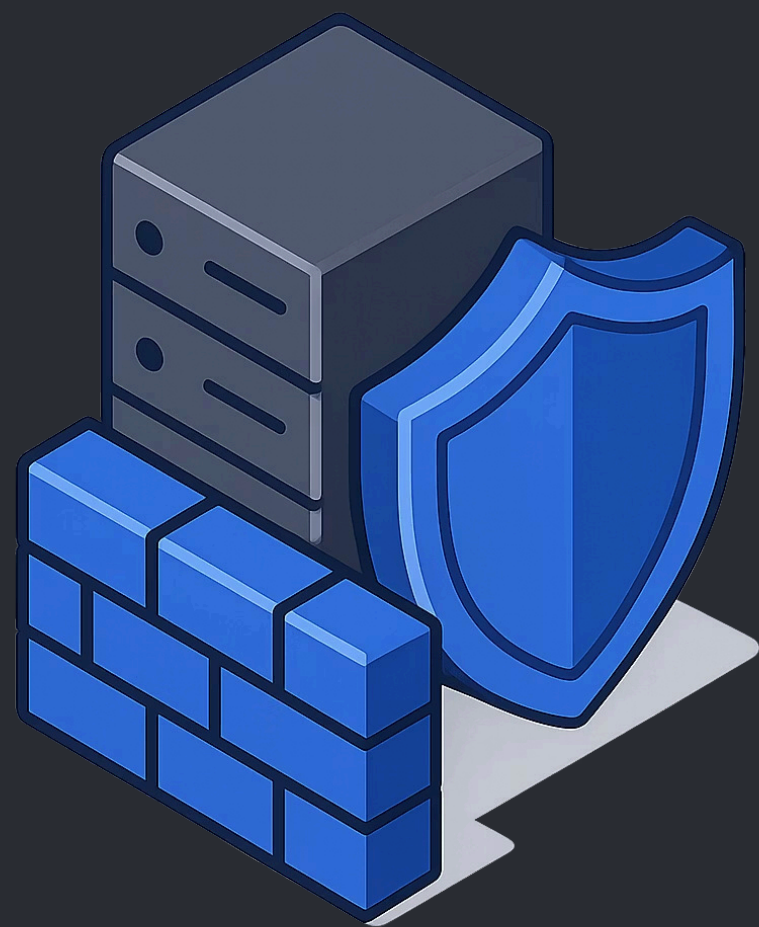
`192.168.1.0/24` # 256 addresses (192.168.1.0 – 192.168.1.255)

`10.0.0.0/16` # 65,536 addresses

`10.0.0.0/8` # 16 million addresses

`172.16.0.0/12` # AWS default VPC range

Firewalls & Security Groups



A **Firewall** is a rules engine that decides which network traffic is *allowed* or *blocked*.

Rules are written as: **Allow TCP traffic on port 443 from any source.**

If no rule allows a packet, it is silently dropped.

In AWS, **Security Groups** act as virtual firewalls attached directly to servers.

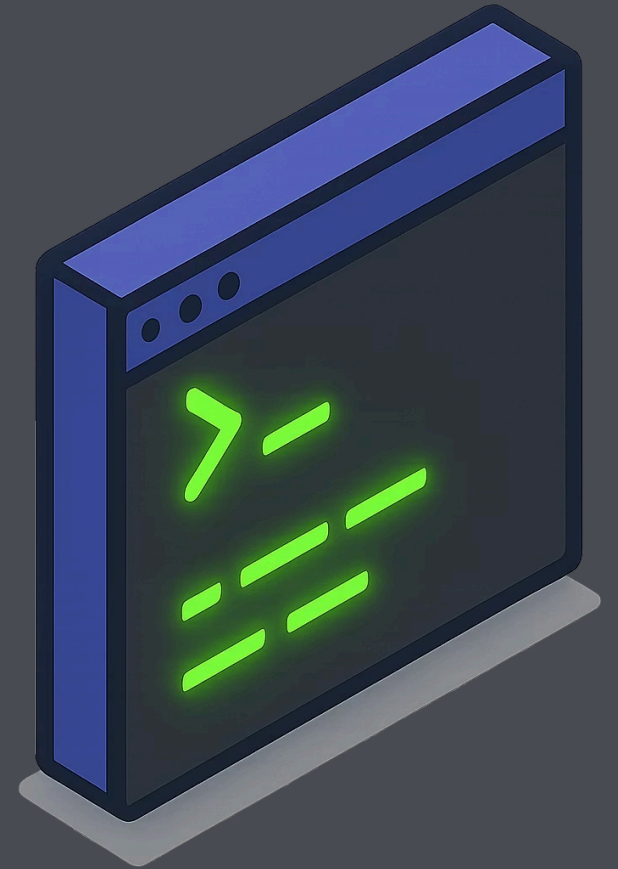
Example Firewall Rules

Action	Protocol	Port	Source
Allow	TCP	443	0.0.0.0/0
Allow	TCP	22	10.0.0.0/8
Deny	ALL	ALL	0.0.0.0/0

PART 2

Linux Command Line Essentials

The 20+ commands every DevOps engineer lives by — from navigating filesystems to debugging live servers.



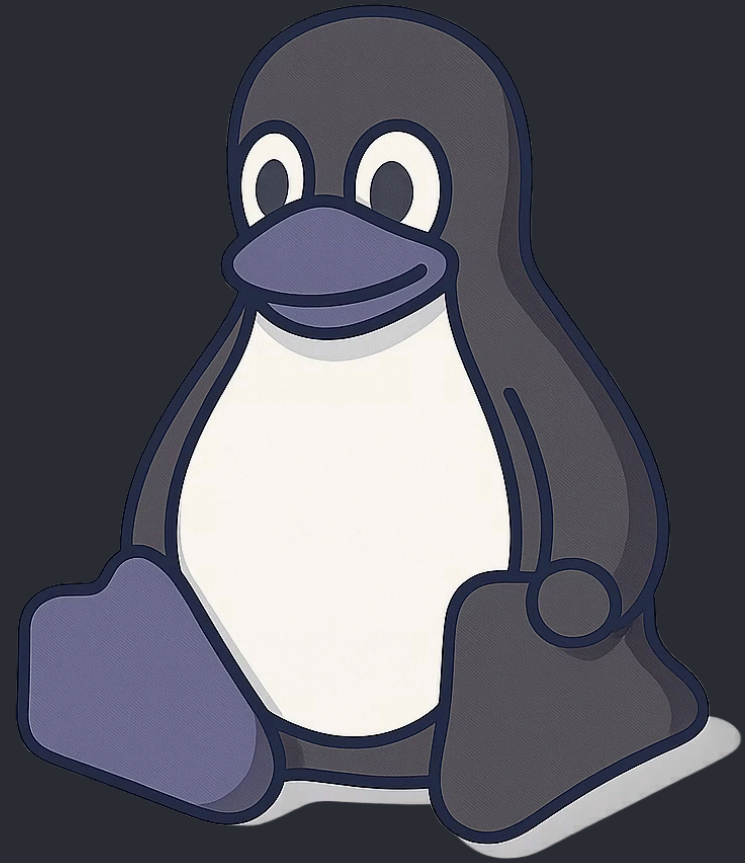
Why Linux?

96%

Cloud Servers

of the world's cloud servers, Docker containers, and Kubernetes nodes run
Linux

As a DevOps engineer, you will spend a significant portion of your career typing commands into a Linux terminal. macOS Terminal is also Unix-based, so almost all commands are identical — giving you a consistent skill set across environments.



Navigating the Filesystem

Essential Navigation Commands

```
pwd          # Print Working Directory: "Where am I?"
ls           # List files in the current directory
ls -la       # List ALL files (including hidden) with details
cd /path/to/dir # Change Directory to an absolute path
cd ..        # Go up one directory
cd ~         # Go to your home directory (/home/username)
tree         # Show a visual tree of directories
```

The Linux Filesystem Layout

/ — The root. The parent of everything.

/home — All user home directories.

/etc — Configuration files.

/var/log — Log files.

/tmp — Temporary files (cleared on reboot).

Reading & Writing Files

```
cat file.txt      # Print full file contents to the terminal
less file.txt     # Scroll through a large file (q to quit)
head -n 20 file.txt # Print only the first 20 lines
tail -n 50 file.txt # Print only the last 50 lines
tail -f app.log   # Follow a log file LIVE (critical for debugging!)
echo "Hello" > file # Write "Hello" to a file (WARNING: overwrites!)
echo "More" >> file # APPEND "More" to a file (safe)
touch newfile.txt # Create an empty new file
```

Manipulating Files & Directories

```
mkdir my-folder      # Create a directory
mkdir -p a/b/c       # Create nested directories in one command
cp file.txt backup.txt # Copy a file
cp -r folder/ backup/ # Copy an entire directory recursively
mv old.txt new.txt    # Rename a file
mv file.txt /var/     # Move a file to another directory
rm file.txt          # Delete a file (PERMANENT - no trash!)
rm -rf my-folder/     # Force delete a directory and all contents
```

 **rm -rf** is one of the most dangerous commands in Linux. Use it with extreme caution!

Searching & Filtering

```
grep "error" app.log      # Search inside a file for the word "error"
grep -r "password" /etc/  # Recursively search all files in /etc
grep -i "Error" app.log   # Case-insensitive search
cat app.log | grep "CRITICAL" # Pipe: filter output of cat through grep
find /var -name "*.log"    # Find all .log files under /var
find . -type f -newer today.txt # Find files newer than a reference file
wc -l app.log              # Count the number of lines in a file
```

The Pipe (|) — Composing Commands

The Pipe character `|` takes the output of one command and sends it as the input to the next.

This allows you to compose complex operations from simple building blocks.

```
# Count how many times "ERROR" appears in a log file
```

```
grep "ERROR" app.log | wc -l
```

```
# See the 10 most disk-consuming folders
```

```
du -sh /var/* | sort -rh | head -10
```

```
# Monitor only Java processes live
```

```
ps aux | grep java
```

File Permissions

Permission Commands

```
ls -la          # Shows permissions as: -rwxr-xr-- (10 characters)
chmod 755 script.sh # Set permissions numerically
chmod +x script.sh # Add executable permission to User
chown user:group file.txt # Change file owner and group
```

Understanding the Permission String: -rwxr-xr--

Position	Character	Meaning
1	-	Type (- = file, d = directory)
2-4	rwx	User (Owner) permissions
5-7	r-x	Group permissions
8-10	r--	Others permissions

Numeric Values: r=4, w=2, x=1

- 755 = User: 7(rwx), Group: 5(r-x), Others: 5(r-x)
- 644 = User: 6(rw-), Group: 4(r--), Others: 4(r--)

Process Management

ps aux	# List every running process from every user
top	# Live, interactive process monitor
htop	# A better-looking top (install separately)
kill 1234	# Gracefully stop process with PID 1234 (SIGTERM)
kill -9 1234	# Forcefully kill process 1234 (SIGKILL - kernel kills it)
pkill nginx	# Kill all processes named "nginx"
sleep 60 &	# Run a command in the background (&)
jobs	# List background jobs in the current terminal session
fg	# Bring a background job to the foreground

Networking Commands

```
ip a                # Show network interfaces and IP addresses
ping 8.8.8.8        # Test connectivity to Google DNS
ping -c 4 google.com # Send exactly 4 pings
curl https://google.com # Fetch HTTP content from a URL
curl -I https://api.example.com # Fetch only HTTP headers
wget https://example.com/file.zip # Download a file
ss -tulwn           # Show which ports/processes are LISTENING
netstat -tulpn       # Alternative to ss (older systems)
traceroute google.com # Show the network path (hops) to a destination
```

Package Management

Ubuntu / Debian — apt

```
sudo apt update      # Refresh the package list from
repositories
sudo apt install nginx # Install the Nginx web server
sudo apt remove nginx  # Uninstall Nginx
sudo apt upgrade       # Upgrade all installed packages
```

RedHat / CentOS — yum / dnf

```
sudo yum install httpd # Install Apache
sudo dnf update         # Update all packages
```

System Information Commands

`uname -a` # Full OS info (kernel version, architecture)

`df -h` # Disk space usage across all filesystems

`du -sh /var/log` # Total size of the /var/log directory

`free -h` # RAM usage (total, used, available)

`uptime` # How long the server has been running

`whoami` # Print the currently logged-in user

`hostname` # Print the machine's hostname

`history` # Show your command history

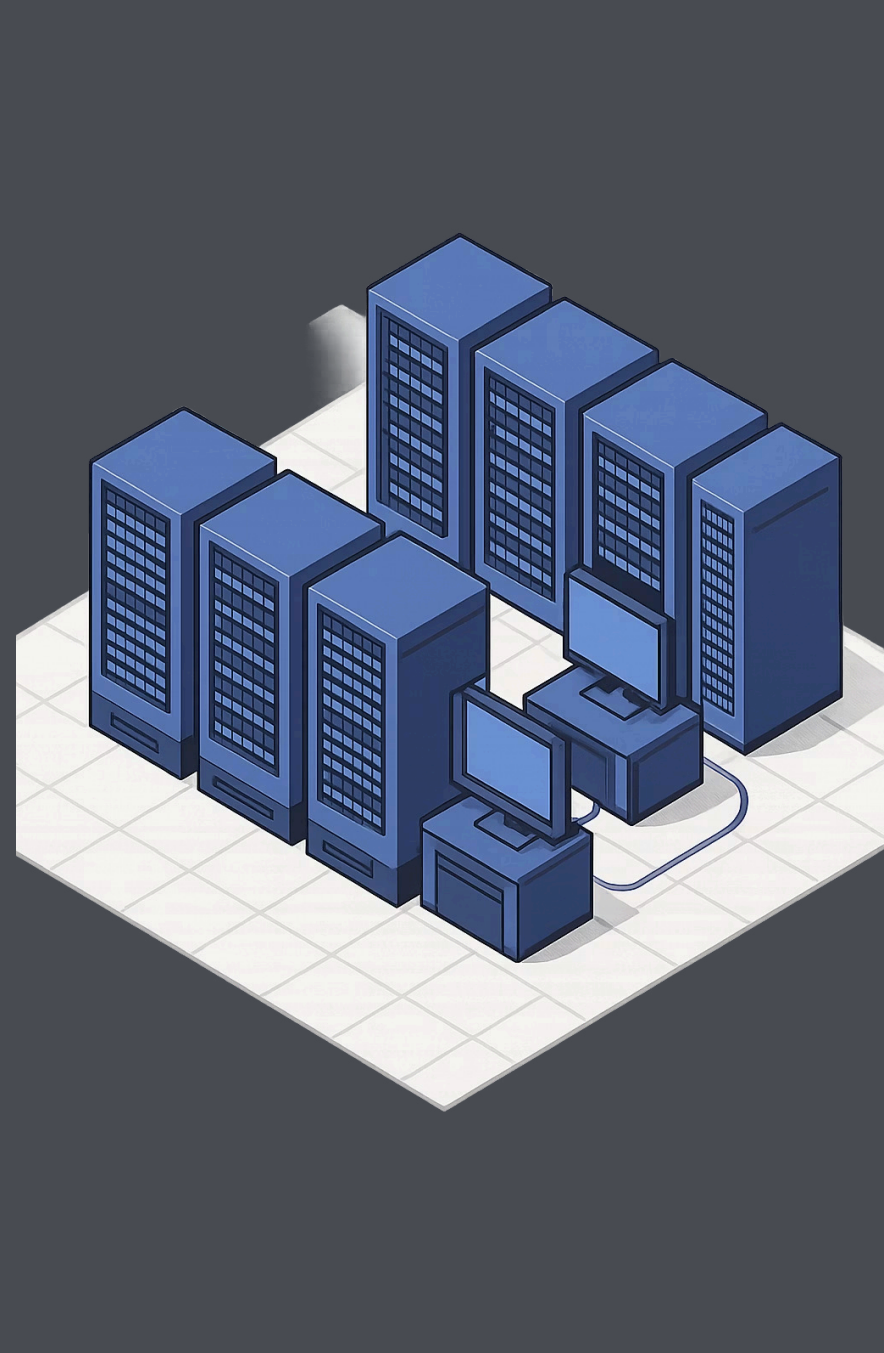
`history | grep ssh` # Find all past SSH commands

SSH: Connecting to Remote Servers

```
# Connect to a remote server
ssh user@192.168.1.100
ssh -i keypair.pem ubuntu@3.88.12.45 # Using a private key file (AWS-style)
ssh -p 2222 user@server.com          # Connect on a non-standard port

# Copy files securely
scp file.txt user@server.com:/var/www/ # Copy a file TO a server
scp user@server.com:/logs/app.log ./   # Copy a file FROM a server

# Create SSH Key Pair (generates id_rsa and id_rsa.pub)
ssh-keygen -t rsa -b 4096
```

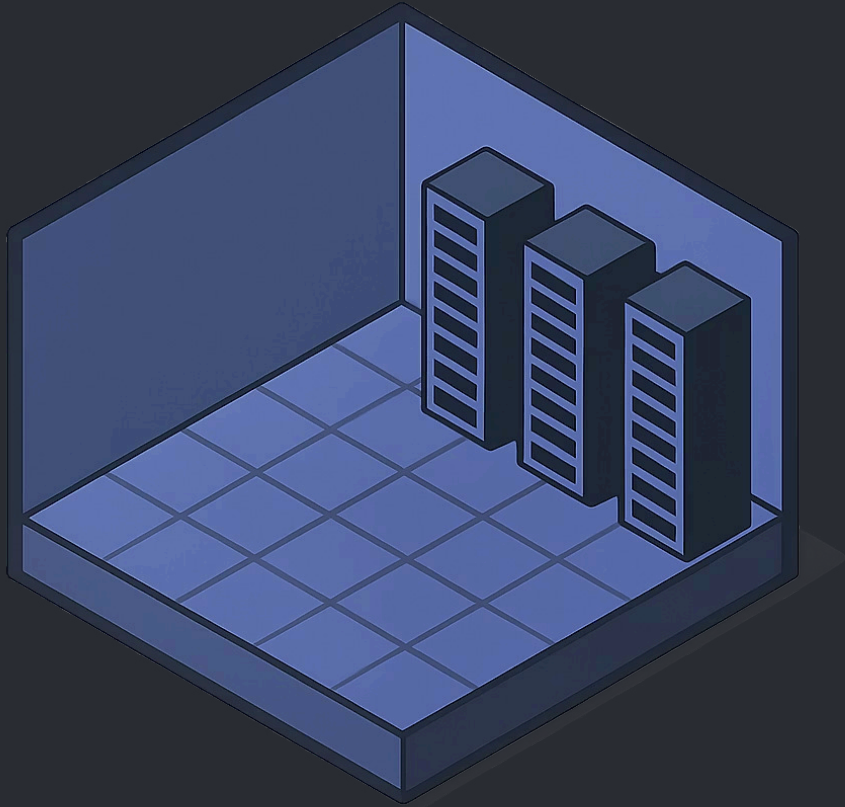



PART 3

Virtualization

How one physical machine became many – and why it changed data centers forever.

The Physical Server Problem



Imagine you buy a powerful physical server with 32 CPU cores and 128GB RAM.

You install just one application on it. Most of the time, this application uses **4% of the CPU** and **8% of the RAM**.

96% of that expensive hardware is completely wasted.

This was the daily reality of data centers in the early 2000s. Server rooms full of expensive machines, each running a single application.

Virtualization to the Rescue

Virtualization is the technology that allows you to split ONE physical server into MANY isolated, independent "virtual" servers. Each Virtual Machine behaves exactly as if it is a physically separate computer. A single powerful host can now run 10, 20, or even more VMs simultaneously, each running a different application.

Isolation

Each VM is completely isolated. One crashing VM doesn't affect others.

Efficiency

Use 80–90% of hardware instead of 4%.

Flexibility

Run Windows and Linux VMs on the same physical machine.

The Hypervisor

The software that makes virtualization possible is called a **Hypervisor**. It sits between the hardware and the Virtual Machines, managing and allocating CPU, Memory, and Storage to each VM.

Two Types

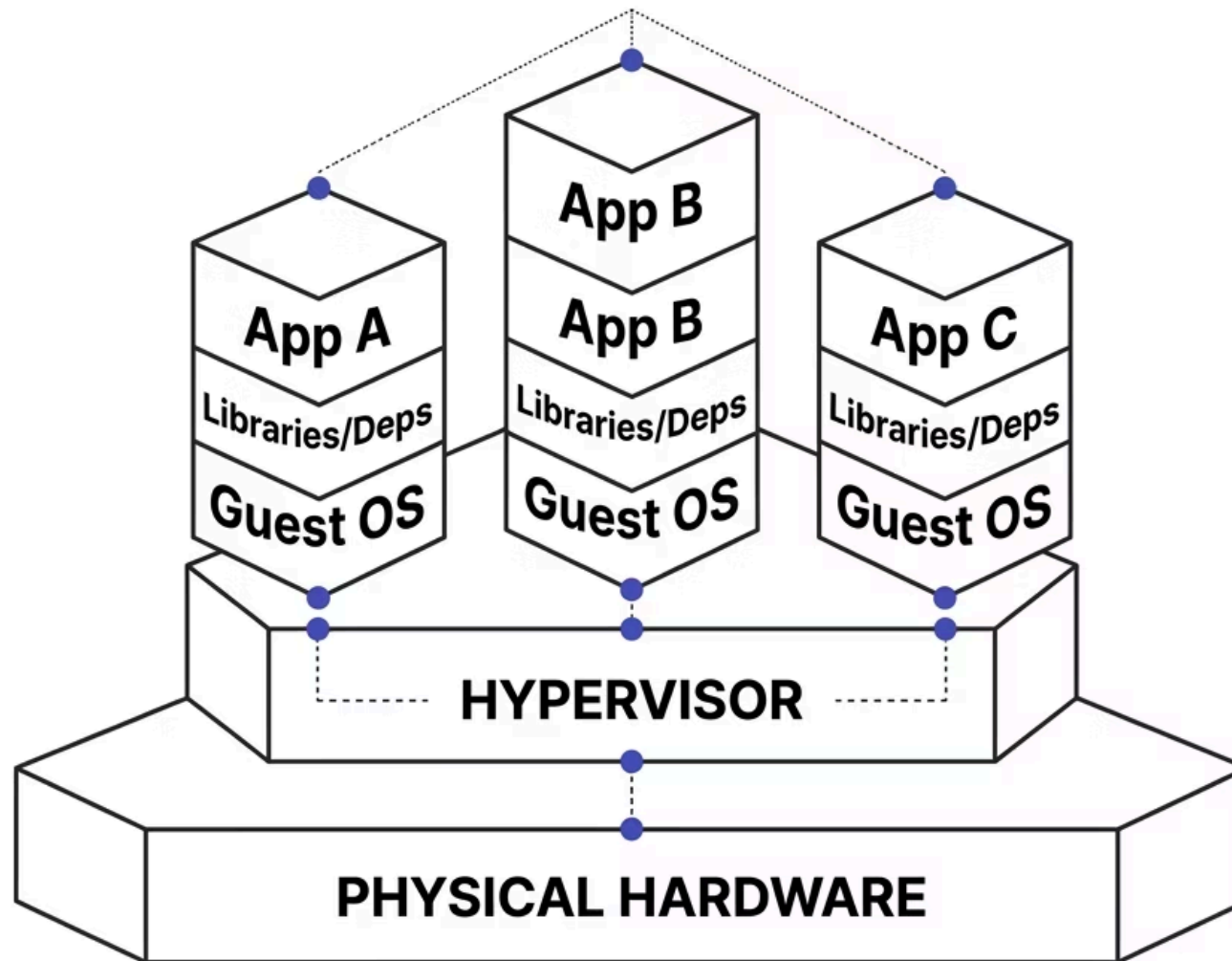
Type 1 — Bare Metal

Runs directly on the hardware. No host OS. Examples: VMware ESXi, Microsoft Hyper-V, KVM. Used in enterprise data centers and AWS.

Type 2 — Hosted

Runs as a normal application on top of an existing OS. Examples: VirtualBox, VMware Workstation. Used on developer laptops.

Virtual Machine Architecture



Each VM carries its own complete **Guest OS** → heavy, gigabytes in size, minutes to boot.

The Remaining Problem with VMs

VMs solved hardware wastage beautifully.

| "It works on my machine!"

A developer writes code on **Ubuntu 20.04** with **Python 3.9**. The production VM runs **RHEL 8** with **Python 3.6**. The code breaks.

VMs don't solve the application **environment consistency** problem.

You still need to manually set up the OS, install dependencies, and configure everything — and pray it matches prod.



PART 3B

Hands-On Lab

Install VMware & Run Ubuntu Desktop

We stop talking theory. You will install a real Linux machine on your Windows laptop right now.

Why VMware Workstation?

VMware Workstation Player is a free Type-2 Hypervisor that runs directly on your Windows laptop.

It allows you to run a completely isolated Ubuntu Linux machine on top of your Windows OS — no dual-boot required.

This is exactly how professional engineers set up local Linux environments for DevOps learning.

Alternatives

VirtualBox — Free & open source.

Works on Windows, Mac, Linux.

UTM — Best option for Mac / Apple Silicon users.

WSL2 — Windows Subsystem for Linux.

Lighter but less isolated.

Step 1 — Download VMware Workstation Player

01

Open your browser and go to:

<https://www.vmware.com/products/workstation-player.html>

02

Click "Download for Free"

03

Choose the Windows version and download the .exe installer.

File size is approximately 600 MB — be patient!

04

Run the .exe file as Administrator once downloaded.



Make sure Virtualization Technology (VT-x) is enabled in your BIOS. Most modern laptops have it enabled by default.

Step 2 — Install VMware Workstation Player

01

When the installer opens, click Next.

02

Accept the License Agreement → click Next.

03

Leave the installation path as default → click Next.

04

Uncheck "Help improve VMware Workstation Player" (optional) → Next.

05

Uncheck "Check for product updates on startup" (optional) → Next.

06

Click Install and wait (~2–3 minutes).

07

When prompted to restart, click Restart Now.

08

After reboot, open VMware Workstation Player from the Start Menu.

Step 3 — Download Ubuntu Desktop ISO

We need an ISO file — this is effectively a digital DVD containing the Ubuntu operating system installer.

01

Go to:
<https://ubuntu.com/download/desktop>

02

Download Ubuntu 22.04 LTS (Long Term Support — stable and widely used).

File size: approximately 5 GB.

03

Save it somewhere easy to find.

e.g. C:\Users\YourName\Downloads\ubuntu-22.04.iso

LTS means Long Term Support — security updates are guaranteed for 5 years. Always use LTS for DevOps work.

Step 4 — Create a New Virtual Machine

01

Open VMware Workstation Player.

03

Select "Installer disc image file (iso)" → Browse to your downloaded Ubuntu ISO file → Next.

VMware detects Ubuntu automatically.

05

Click Next.

02

Click "Create a New Virtual Machine".

04

Fill in your details:

Full Name: devops-student | Username: devops (all lowercase, no spaces) |

Password: Something you will remember

Step 5 — Allocate Resources to the VM

01

Virtual Machine Name: Ubuntu-DevOps-Lab

02

Location: Keep default or choose a folder on your fastest drive. → Click Next.

03

Disk Size: Set to 30 GB minimum (40 GB recommended for Docker labs).

Select "Store virtual disk as a single file" for better performance.

04

Click Next → then Customize Hardware.

Step 6 — Customize VM Hardware

In the Customize Hardware window:

Resource	Recommended Setting
Memory (RAM)	4096 MB (4 GB) minimum — 8192 MB (8 GB) ideal
Processors	2 CPU cores minimum — 4 cores ideal
Network Adapter	Leave as NAT (shares your laptop's internet)
Display	Enable 3D acceleration for smoother GUI

- Click Close → then Finish.
- VMware will now boot the VM and Ubuntu will auto-install itself (Easy Install mode).
- This takes approximately 5–10 minutes.

Step 7 — First Boot & Setup

- After Ubuntu finishes installing, the VM restarts automatically.
- You will be greeted by the Ubuntu login screen.
- Enter the password you set in Step 4. You are now inside a full Ubuntu 22.04 Desktop environment running on your Windows laptop!

Install VMware Tools (Critical)

VMware Tools improves integration (shared clipboard, full-screen support, drag-and-drop):

```
# Open the Ubuntu terminal (Ctrl + Alt + T) and run:  
sudo apt update  
sudo apt install -y open-vm-tools open-vm-tools-desktop  
sudo reboot
```

Step 8 — First Commands Inside Your VM

Open the Terminal inside Ubuntu (press Ctrl + Alt + T):

```
# Check Ubuntu version
```

```
cat /etc/os-release
```

```
# Update the system
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Check available disk space
```

```
df -h
```

```
# Check RAM
```

```
free -h
```

```
# Verify network connectivity
```

```
ping -c 3 google.com
```

 Congratulations! You now have a real Linux machine running inside your Windows laptop. This is your DevOps lab environment for the rest of the course!

VM Snapshot — Your Safety Net

A Snapshot saves the exact current state of the entire VM at a specific point in time.

If you accidentally break your Ubuntu environment (wrong `rm -rf`, bad config), you can instantly restore to the snapshot.

Creating a Snapshot

01

With the VM running, go to Player menu → Manage → Take Snapshot.

02

Name it: Clean-Ubuntu-Base

03

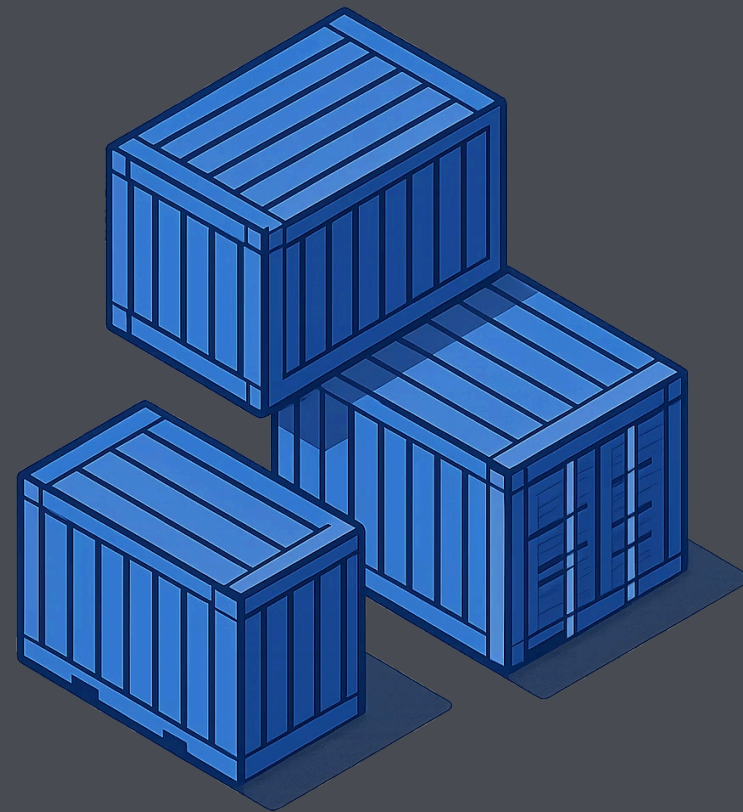
Click Take Snapshot.

Always take a snapshot before trying something risky in your lab environment. It is completely free and takes less than 30 seconds.

PART 4

Containerization

Lightweight, portable, and consistent — containers changed how software is built and deployed.



The Idea: Isolation Without a Full OS

The Big Question

What if, instead of virtualizing the entire hardware (including a full Guest OS), we could isolate applications at the process level, sharing the Host OS kernel?

This is exactly what containers do.

Two core Linux kernel features make this possible:

Namespaces

Provide **isolation** (process, network, filesystem). Each container thinks it has its own private server.

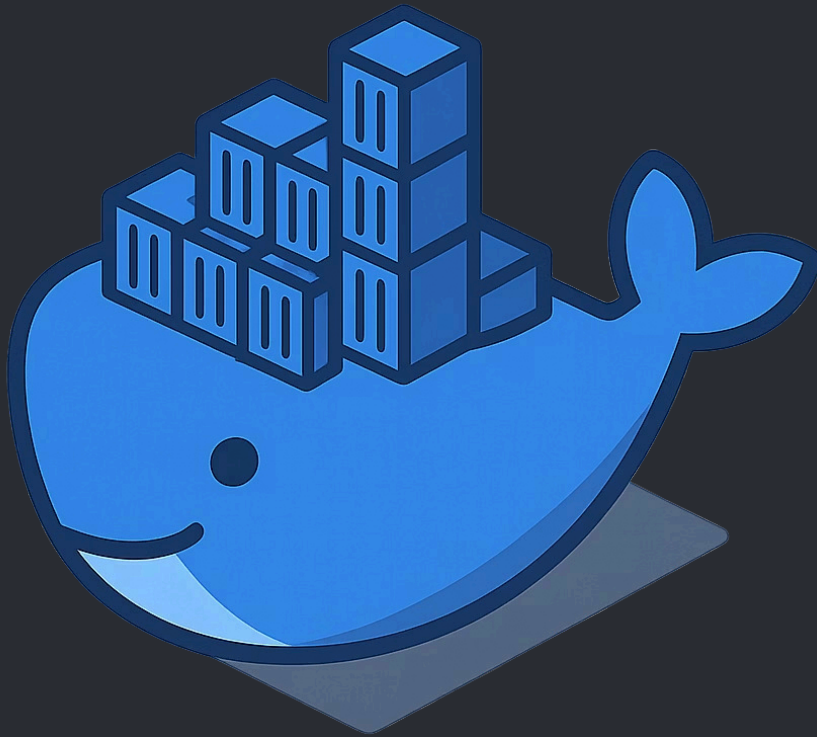
cgroups (Control Groups)

Limit and measure resource usage (CPU, memory) so one container cannot starve others.

VMs vs Containers: Side by Side

Feature	Virtual Machine	Container
Size	Gigabytes	Megabytes
Boot Time	Minutes	Milliseconds
OS	Full Guest OS per VM	Shares Host Kernel
Isolation	Hardware Level	Process Level
Portability	Complex to move	Just docker push
Use Case	Legacy apps, strong isolation needed	Modern microservices, CI/CD

What is Docker?



Docker is the platform that packaged containerization technology into a simple, useable tool.

Before Docker (2013), using Linux namespaces and cgroups directly was extremely complex and required deep kernel expertise.

Docker made it trivially simple for any developer to run: `docker run my-app`.

Docker's three key contributions

01

Simple CLI

Wraps complex Linux kernel features behind friendly commands.

02

Dockerfile

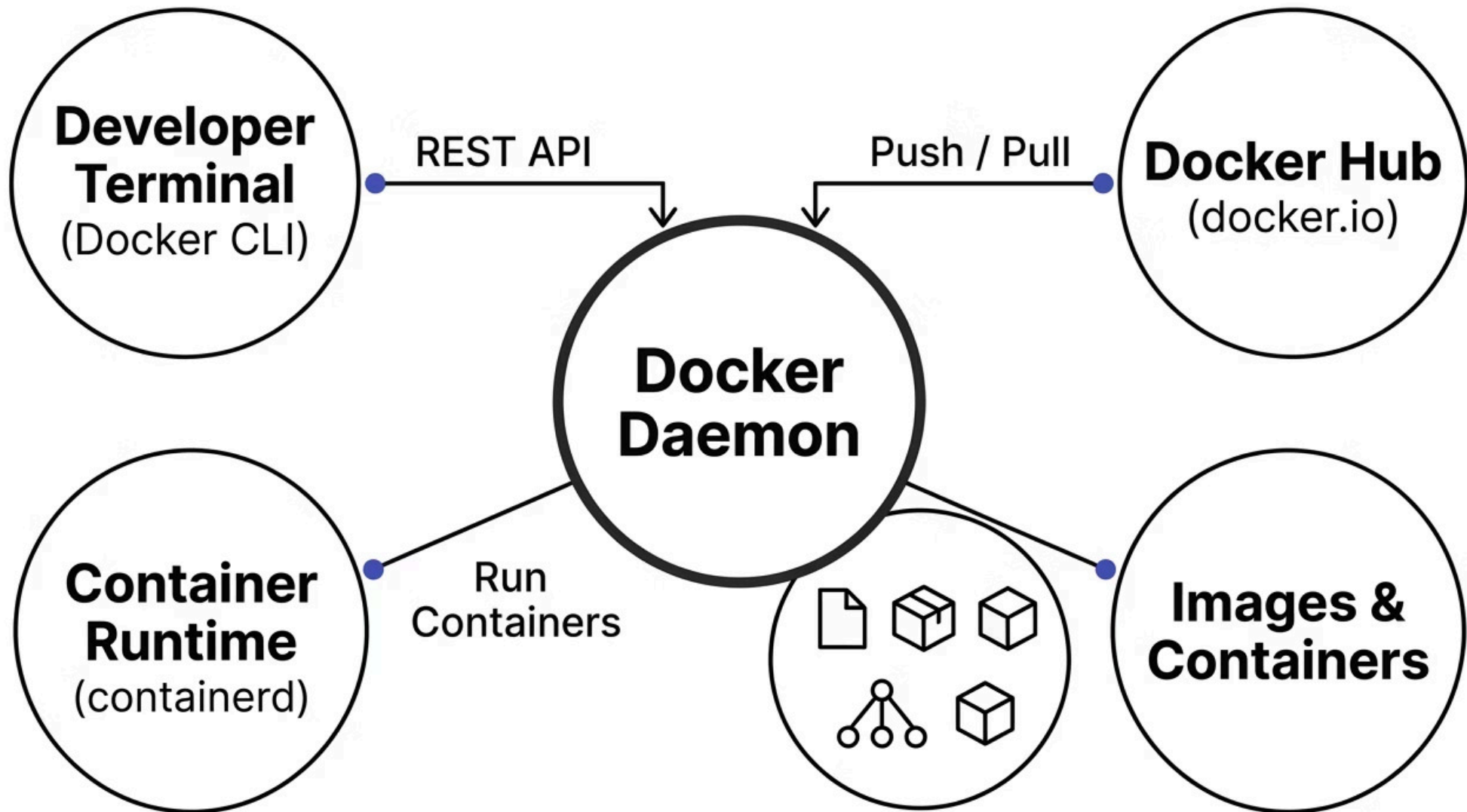
A simple text format to describe and reproducibly build any environment.

03

Docker Hub

A centralized public registry to share images globally.

Docker Architecture



The **Docker CLI** is what you type into. It talks to the **Docker Daemon** via a REST API. The daemon manages images, containers, networks, and volumes. Images are pulled from **Docker Hub** and run by the container runtime **containerd**.

Docker Images vs Containers

The Class/Object Analogy

Docker Image = Blueprint (Class)

A Docker Image is like a Class in programming. It is a static, read-only blueprint. It defines the environment.

```
# Image = Blueprint  
docker images
```

Docker Container = Running Instance (Object)

A Docker Container is like an Object (instance) created from that Class. It is a live, running process.

From 1 single Image, you can run 100 simultaneous Containers.

```
# Container = Running Instance of that Blueprint  
docker ps
```

The Docker Image: A Layered Cake

Every instruction in a Dockerfile creates a new **immutable layer** on top of the previous one.

Layers that haven't changed are **cached**. Subsequent builds reuse cached layers — making them dramatically fast.

Layer 5: **COPY** app.py . ← Changes often (your code)

Layer 4: RUN pip install ← Changes rarely (dependencies)

Layer 3: **COPY** requirements.txt ← Changes rarely

Layer 2: RUN apt-get update ← Almost never changes

Layer 1: **FROM** python:3.11-slim ← The base

Writing Your First Dockerfile

1. Choose the base image from Docker Hub

```
FROM python:3.11-slim
```

2. Set the working directory inside the container

```
WORKDIR /app
```

3. Copy dependency file first (enables layer caching)

```
COPY requirements.txt .
```

4. Install dependencies (cached unless requirements.txt changes)

```
RUN pip install -r requirements.txt
```

5. Copy the rest of your application code

```
COPY . .
```

6. Expose the port the app listens on (documentation only)

```
EXPOSE 5000
```

7. The command to run when the container starts

```
CMD ["python", "app.py"]
```

FROM

Sets the base image. Always start here.

WORKDIR

Sets the working directory inside the container.

COPY

Copies files from your machine into the image.

RUN

Executes a shell command during the build.

EXPOSE

Documents which port the app uses.

CMD

The default command to run when the container starts.

Essential Docker Commands

Image Management

```
docker images          # List locally stored images
docker pull nginx:latest # Download an image from Docker Hub
docker build -t myapp:1.0 . # Build from Dockerfile in current dir
docker rmi myapp:1.0    # Remove a local image
```

Container Lifecycle

```
docker run -d -p 8080:80 --name web nginx # Create & start a container
docker run -it ubuntu:22.04 /bin/bash # Interactive terminal
docker ps # List running containers
docker ps -a # List ALL containers (inc. stopped)
docker stop web # Gracefully stop a container
docker start web # Start a stopped container
docker rm web # Delete a stopped container
docker exec -it web /bin/bash # Open a shell INSIDE a running container
docker logs -f web # Stream logs from a container
```

Port Mapping Deep Dive

By default, a container is **completely isolated** — nobody can reach it from outside.

-p 8080:80 creates a tunnel: traffic hitting your laptop at port 8080 is forwarded into the container at port 80.

```
docker run -d -p 8080:80 nginx
#           ^^^^  ^^
#   Host Port Container Port
```

You can map any host port to any container port:

```
docker run -d -p 3000:5000 myapp:1.0
# Access via: localhost:3000 → maps to → container's port 5000
```

Environment Variables in Docker

Applications need configuration (database URLs, API keys, etc.). **Never bake secrets into a Docker Image!** Pass them at runtime using environment variables.

```
docker run -d \  
-e DB_HOST=postgres.example.com \  
-e DB_PASSWORD=supersecret \  
-p 8080:80 \  
myapp:1.0
```

Inside the container, these are accessible like any shell variable:

```
import os  
db_host = os.environ.get("DB_HOST")
```

Docker Cleanup Commands

Docker aggressively caches images and keeps stopped containers, consuming disk space rapidly.

```
docker system df      # See how much disk Docker is using
docker system prune   # Delete all stopped containers, unused networks, dangling images
docker system prune -a # Also delete unused images (not just dangling)
docker volume prune   # Delete all unused volumes
```

RECAP

What We Covered Today

Five foundational pillars every DevOps engineer needs to understand — from the first packet to the running container.



Summary

What We Covered Today



Networking

Every server has an IP. Every service has a Port. DNS translates names to IPs.



Linux

20+ essential commands to navigate, manage, and debug Linux servers.



Virtualization

Hypervisors split one physical machine into many isolated VMs.



Containers

Share the OS kernel for lightweight, fast, portable isolation.



Docker

The platform that made containers mainstream with a simple CLI, Dockerfile format, and global registry.

Next Steps

Before Day 3

- ☐ Install Docker Desktop on your machine.
- ☐ Complete all 3 labs covering Linux, Docker Containers, and Docker Images.
- ☐ Read over the Dockerfile Best Practices official documentation.

Day 3 Preview

- Docker Compose for multi-container apps
- Volumes and persistent storage
- Introduction to Kubernetes

Quick Reference: Key Port Numbers

Port	Protocol	Common Use
22	SSH	Secure remote terminal access
80	HTTP	Unencrypted web traffic
443	HTTPS	Encrypted web traffic (TLS)
3000	HTTP (custom)	Node.js / React dev servers
3306	MySQL	MySQL database
5000	HTTP (custom)	Flask / Python apps
5432	PostgreSQL	PostgreSQL database
6379	Redis	Redis in-memory cache
8080	HTTP (custom)	Alternative web / proxy port
27017	MongoDB	MongoDB database

Quick Reference: Essential Linux Commands

Navigation & Files

`pwd` / `ls -la` / `cd` / `tree`
`cat` / `less` / `tail -f`
`cp` / `mv` / `rm -rf` / `mkdir -p`
`grep -r` / `find` / `wc -l`

Permissions & Processes

`chmod 755` / `chown user:group`
`ps aux` / `top` / `kill` / `kill -9`
`jobs` / `fg` / `&`

Network & System

`ip a` / `ping` / `curl -I`
`ss -tulwn` / `traceroute`
`ssh` / `scp` / `ssh-keygen`

System Info

`df -h` / `free -h` / `uname -a`
`du -sh` / `uptime` / `history`

Package Management

`sudo apt update` && `apt install`
`sudo yum install` / `dnf update`

Quick Reference: Essential Docker Commands

Images

```
docker images
docker pull nginx:latest
docker build -t myapp:1.0 .
docker rmi myapp:1.0
```

Containers

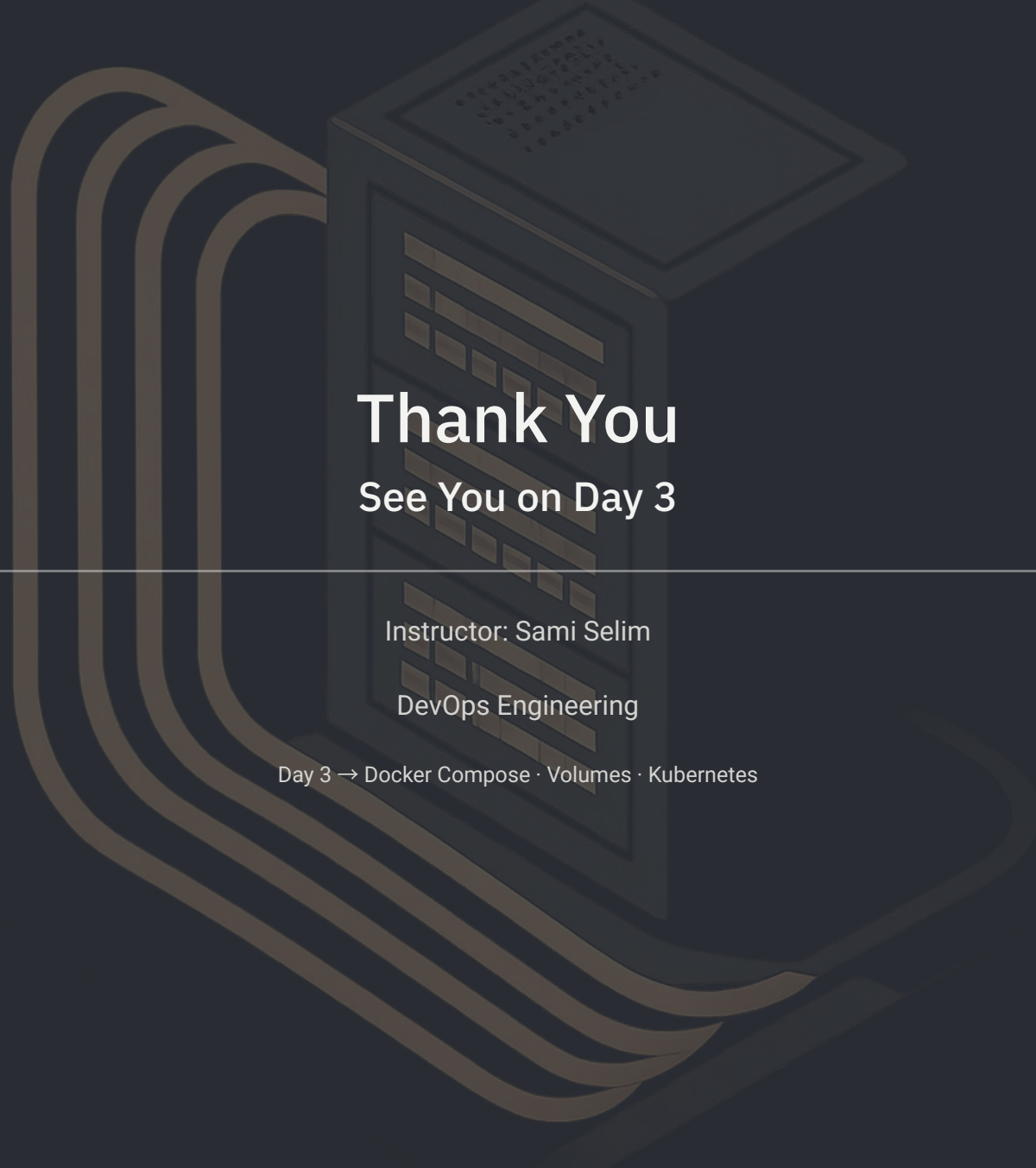
```
docker run -d -p 8080:80 --name web nginx
docker run -it ubuntu:22.04 /bin/bash
docker ps / docker ps -a
docker stop web / docker start web
docker rm web
```

Inspect & Debug

```
docker exec -it web /bin/bash
docker logs -f web
```

Cleanup

```
docker system df
docker system prune
docker system prune -a
docker volume prune
```



Thank You

See You on Day 3

Instructor: Sami Selim

DevOps Engineering

Day 3 → Docker Compose · Volumes · Kubernetes